

A MATHEMATICAL THEORY OF REUSABLE NEURAL BASES FOR NETWORK COMPRESSION

Binshuai Wang, Peng Wei

Department of Computer Science
The George Washington University
Washington D.C., USA
{derekwang, pwei}@gwu.edu

ABSTRACT

As large AI models become increasingly prevalent across a wide range of applications, memory cost has become a critical bottleneck in both training and inference. To mitigate this issue, we introduce the Linear Reusable Neural Bases Architecture (LRNBA), a novel framework aimed at improving parameter efficiency and reducing memory cost. Inspired by recurrent neural network (RNN) designs, we introduce the notion of neural basis, and the core idea of our approach is to represent each network block as a linear combination of a shared set of such neural bases, thereby enjoying highly network compression rate while maintaining stable training. By defining virtual “neurons” through the reuse of these bases, the proposed architecture enables the construction of substantially wider and deeper networks under the same parameter budget. Extensive experiments demonstrate that our model achieves comparable or even faster convergence and lower loss than classical architectures, while maintaining stable training dynamics.

1 INTRODUCTION

Large AI models have attracted increasing attention in the machine learning community, since they outperform smaller models across a broad range of domains (Brown et al., 2020; Hoffmann et al., 2022). As indicated by empirical scaling laws (Kaplan et al., 2020; Hoffmann et al., 2022), increasing the number of model parameters can substantially improve the approximation ability of neural networks. In alignment with these observations, modern AI models are often designed at increasingly large scales, containing billions or even trillions of parameters and requiring hundreds or thousands of GPUs or TPUs for training and deployment (Brown et al., 2020; Fedus et al., 2022).

Although large AI models have demonstrated remarkable performance, training and deploying them remain challenging, especially for researchers and business owners who face substantial limitations in computational resources. One of the major bottlenecks is the memory requirement. Training or deploying a large-scale model often requires hundreds or even thousands of GPUs operating simultaneously, while the high price of GPUs and TPUs further increases the overall financial burden (Narayanan et al., 2021). Overall, these resource and hardware requirements limit the ability of many researchers and organizations to further develop and deploy large AI models.

A promising approach to addressing this challenge is to reuse the same neural network modules multiple times, thereby reducing the number of distinct parameters and the associated memory cost. This idea is closely related to recurrent neural networks (RNNs), which repeatedly apply a shared transformation across different time steps or positions (Elman, 1990; Jordan, 1997). Moreover, following the philosophy that “compression is intelligence”, networks that share parameters across blocks should attain higher compression than those with independent parameters, and thereby typically exhibit better generalization ability. However, classical RNNs often suffer from optimization difficulties, as repeated transformations can result in unstable gradient propagation, including vanishing or exploding gradients. Consequently, many large-scale AI models are commonly designed with independent layers and integrated only limited reuse mechanisms for internal components.

In light of these observations, a natural question arises:

Can we design a architecture that intergrates the reuse mechanism of RNNs to improve parameter efficiency and reduce memory costs, while maintaining training stability and comparable performance?

Inspired by the theory of linear bases in linear algebra and sinusoidal bases in Fourier analysis, we propose a novel model architecture, the Linear Reusable Neural Bases Architecture (LRNBA), which implements an internal reuse mechanism to improve parameter efficiency and reduce memory cost. By dynamically representing each network block as a linear combination of reusable neural bases, our model allows the same neurons to be reused multiple times, thereby dramatically reducing the parameter count while maintaining training stability and achieving comparable or even better performance. Moreover, to further enhance parameter efficiency, we extend the neural bases concept to other types of network components, including attention and projection blocks, and introduce a technique for sub-neural reuse. Experimental results indicate that our architecture achieves comparable or even faster convergence and a lower final loss than the classical architectures, while maintaining stable training and a substantially reduced parameter count.

Contributions. The main contributions of this work are summarized as follows:

- (a) Grounded in the principles of linearity and reuse, we introduce a simple yet effective architecture, the Linear Reusable Neural Bases Architecture (LRNBA), designed to improve parameter and neuron efficiency while reducing memory costs. Compared to classical neural networks with fully independent parameters, our model requires less memory while maintaining stable training and achieving comparable or even better training performance and generalization.
- (b) We introduce a novel elementary concept in neural network design—the neural basis—along with a geometric interpretation from the perspective of vector fields. This concept is core to our theoretical framework, and we extend it to other network components, including attention neural bases and projection neural bases. Furthermore, we also present the notion of sub-neural reuse along with its associated implementation techniques.
- (c) Unlike classical models, whose parameter count depends on both depth and width, the parameter count of our proposed model is almost decoupled from the depth dimension and depends primarily on the width dimension. This enables the design of substantially wider and deeper networks under a limited memory budget, while providing more flexibility to accommodate practical requirements. Furthermore, our experiments show that depth expansion is more critical than width expansion for achieving high precision, and our architecture naturally supports the construction of deeper models.

Organization. The remainder of this paper is organized as follows. Section 2 provides a basic review of related work. Section 3 revisits several key concepts and formulations in deep learning theory. Section 4 details our proposed architecture, the associated training strategy, and further improvements to the reuse mechanism and parameter efficiency. Section 5 discusses the advantages and limitations of our model. Section 6 presents several computational experiments to validate our model.

Notations. The function $\sigma(\cdot)$ represents an element-wise nonlinear activation, $\phi(\cdot)$ denotes a neural basis function, and $R(\cdot)$ denotes a residual block. The notation $\langle \cdot, \cdot \rangle$ represents the inner product, and α denotes a scalar coefficient. Let d denote the embedding dimension, m the width (i.e., hidden dimension or number of neurons), L the network depth, H the number of attention heads, and T the sequence length of tokens. For a residual feedforward block, (S, b) denotes the sensor parameters, while P denotes the response parameters. For a single neural basis, or the j -th neuron, (s_j, b_j) denotes the sensor parameters, while p_j denotes the response parameters.

2 RELATED WORK

Several lines of research are closely related to our work in terms of parameter reuse. We organize these studies into two main categories: recurrent neural networks (RNNs) and parameter-sharing methods beyond recurrent architectures.

Recurrent Neural Networks. Recurrent neural networks (RNNs) represent one of the most widely adopted forms of parameter sharing and reuse in neural architectures. Classical recurrent architectures

include the Elman network, the Jordan network, long short-term memory networks (LSTMs), and gated recurrent units (GRUs) (Elman, 1990; Jordan, 1997; Hochreiter & Schmidhuber, 1997; Cho et al., 2014). Other approaches improve parameter efficiency through low-rank factorizations and shared low-rank components (Lu et al., 2016), or use auxiliary networks, such as HyperNEAT, to generate related parameters across layers or time steps (Ha et al., 2016).

Despite their parameter efficiency, standard RNNs are difficult to optimize because repeated multiplication through time can lead to vanishing or exploding gradients during backpropagation through time (Hochreiter & Schmidhuber, 1997; Bengio et al., 1994; Pascanu et al., 2013). LSTMs and GRUs mitigate this problem through gating. Other approaches improve stability through orthogonal recurrent transformations (Helfrich et al., 2018), residual or convolutional recurrent structures (Bai et al., 2019), and residual scaling techniques such as ReZero (Bachlechner et al., 2021).

Parameter Sharing Beyond RNNs. Parameter reuse has also been explored beyond recurrent architectures. Tensorized neural networks compress dense transformations using tensor-train decompositions (Novikov et al., 2015), while related methods employ structured matrices, such as Toeplitz-like, Hankel-like, and block-circulant matrices, to reduce storage and computation costs (Sindhwani et al., 2015).

Parameter sharing is also widely studied in Transformers (Vaswani et al., 2017). Pires et al. (2023) examine redundancy among Transformer feed-forward network (FFN) blocks and show that a single wide FFN module can be shared across multiple layers. The Universal Transformer repeatedly applies shared self-attention and transition blocks across depth (Dehghani et al., 2019). The Sparse Universal Transformer further combines depth-wise sharing with sparse mixture-of-experts layers to increase capacity without activating all parameters at every iteration (Tan et al., 2023). ALBERT adopts cross-layer parameter sharing among Transformer encoders, substantially reducing the number of independently learned parameters while maintaining competitive performance (Lan et al., 2020).

More recently, looped Transformers repeatedly apply a fixed block, or a small set of blocks, to an evolving hidden representation. Such models can emulate iterative computational procedures (Gian-nou et al., 2023) and provide a useful inductive bias for learning iterative algorithms and in-context tasks with fewer parameters (Yang et al., 2023).

Connection to Our Work. Our model can be viewed as an RNN-like architecture, since it reuses the same collection of neurons across multiple steps. In contrast to classical RNNs that apply an identical transformation repeatedly, and other approaches that focus on reuse at the component level, our method explicitly reuses nonlinear features at the *neuron level* across depths, including both attention and FFN components, providing more flexible combinations and better parameter efficiency. Consequently, it achieves better generalization while maintaining stable training.

3 BACKGROUND

Before presenting our main results, we briefly review several key concepts and formulations.

3.1 DEEP LEARNING

Deep learning improves the representational ability of neural networks through the use of sequentially composed architectures (LeCun et al., 2015). By hierarchically composing multiple nonlinear transformations, deep neural networks can represent certain functions more efficiently than the shallow networks (Raghu et al., 2017). Consequently, deep networks often exhibit better approximation ability than the shallow networks for complex learning tasks.

Although the sequentially composed architecture of deep neural networks enables models to learn more complex features and representations, it also introduces substantially higher computational and memory costs during training or inference, compared to shallow networks (Rajbhandari et al., 2020). Since each component must be processed sequentially, a large number of parameters and intermediate activations must be held in memory at all times, even though only a small fraction are actively executing during training or inference. This memory retaining grows with network depth, posing a significant challenge for researchers with limited computational resources.

3.2 RESNET NETWORKS

Residual networks (ResNets) play an important role in the deep learning area, as they alleviate the vanishing-gradient problem and enable the stable training of very deep networks (He et al., 2015). The key feature of a ResNet is the use of shortcut connections, which allows the input information to be passed directly to deeper layers rather than propagating only through nonlinear layers.

For example, a residual feedforward block in a ResNet can be written as

$$x + R(x),$$

where x represents the shortcut connection and $R(x)$ represents the residual component. A typical residual component takes the form

$$R(x) = P\sigma(Sx + b),$$

where $\sigma(\cdot)$ is an element-wise nonlinear activation function, and (S, b) are the learnable parameters of the input affine layer, and P are the learnable parameters of the output linear layer.¹

From a more detailed perspective, the residual component can be viewed as a linear combination of a shared collection of nonlinear functions. The parameters (S, b) determine the directions and intercepts (offsets) of these nonlinear functions in the input space, while P specifies the coefficients used to combine them in the output space. We further extend this viewpoint in Section 4 to develop a new model architecture that reuses parameters to reduce memory costs.

4 METHODOLOGY

In this section, we introduce our novel architecture, the Linear Reusable Neural Bases Architecture (LRNBA), designed to improve parameter efficiency and reduce memory cost. The key idea is to reuse a shared collection of neural bases while assigning different coefficients to construct blocks at different positions, thereby allowing the same neurons to be reused multiple times. In the remainder of this section, we first define neural bases and provide a geometric interpretation from the perspective of vector fields. We then describe the model architecture, training strategy, and key implementation details.

4.1 THE NEURAL BASES

In this subsection, we first introduce our core concept of neural bases and provide a geometric interpretation via vector fields.

Definition of Neural Bases. As introduced in section 3, a residual feedforward block has the form

$$x + P\sigma(Sx + b). \quad (1)$$

Let m denote the hidden dimension of the residual block, and write the rows of S , the entries of b , and the columns of P as

$$S^\top = [s_1, s_2, \dots, s_m], \quad b^\top = [b_1, b_2, \dots, b_m], \quad P = [p_1, p_2, \dots, p_m],$$

where s_j is the j -th row of S , b_j is the j -th entry of b , and p_j is the j -th column of P .

For each $j = 1, 2, \dots, m$, we define the function $\phi_j(x)$ as:

$$\phi_j(x) := \sigma(\langle x, s_j \rangle + b_j)p_j.$$

Then the residual component of the network can be rewritten as a summation of such functions:

$$P\sigma(Sx + b) = \sum_{j=1}^m \sigma(\langle x, s_j \rangle + b_j)p_j = \sum_{j=1}^m \phi_j(x). \quad (2)$$

¹For simplicity, we omit the bias term in the output layer. This omission does not affect the nonlinear structure of the residual component, since the output bias contributes only a constant translation.

This formulation reveals that the mapping of the residual block can be further decomposed into a group of basic nonlinear functions $\{\phi_j\}_{j=1}^m$. Based on this decomposition, and since each of these functions serve as an *atomic* nonlinear structure from the input to the output space—in the sense of being the simplest and independent building block—we refer to each function ϕ_j as a *neuron* or *neural basis* of the network, analogous to a sinusoidal basis in Fourier analysis or a basis vector in linear algebra. Moreover, we refer to the associated parameters (s_j, b_j) for the input space as the *sensor* of neuron ϕ_j and p_j as the *response* of neuron ϕ_j .

Superposition of Vector Fields. Here, we provide a geometric interpretation of the neural basis. Specifically, each neural basis can be viewed as a vector field from the input space to the output space. For the j -th neuron, as illustrated in Figure 1a, the sensor vector s_j and scalar b_j specifies the direction and bias of the input vector field, and vector p_j determine the response vector in the output vector field. These parameters are connected through a bilinear structure between the nonlinear activation $\sigma(\langle x, s_j \rangle + b_j)$ and the response vector p_j . As a result, a residual feedforward block can be viewed as a superposition of many such vector fields, as depicted in Figure 1b.

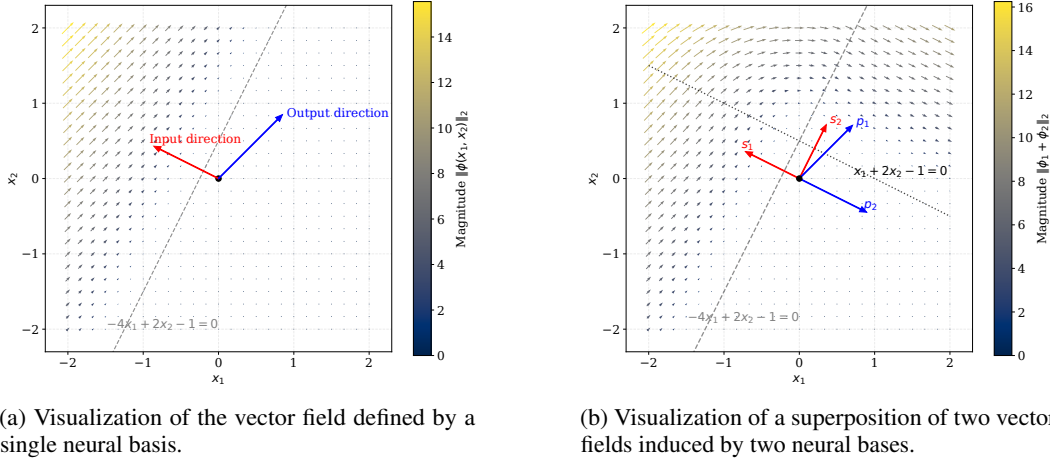


Figure 1: Geometric interpretation of neural bases from the perspective of vector fields. **(a)** Visualization of a single neural basis, defined by the function $\phi(x) = \sigma(\langle x, s \rangle + b)p$ with parameters $s = (-4, 2)$, $b = -1$, and $p = (1, 1)$. The red and blue arrows indicate the directions of sensor s and response p in the input and output vector fields, respectively. The dashed line marks the decision boundary induced by the equation $\langle x, s \rangle + b = 0$. The small arrows on the grid illustrate the push-forward effect of the vector field. **(b)** A superposition of two vector fields, where the first neural basis is the same as that shown in **(a)** and the second basis has parameters $s_2 = (1, 2)$, $b_2 = -1$, and $p_2 = (2, -1)$. Note that a single neuron can only cause one directional effect, whereas the combination of two neurons can generate multiple directional effects, as exhibited by the rotational dynamics in **(b)**.

Attention Neural Bases and Projection Neural Bases. Beyond residual feedforward blocks, the concept of neural bases is also extendable to other types of neural network components, such as attention neural bases and projection neural bases. Due to space limitations, detailed formulations are provided in Appendix A.

The above generalized perspective enables us to consistently treat different types of network blocks as linear combinations of corresponding neural bases, extending the idea of reusable neural bases to a broader range of neural network architectures, including transformers.

4.2 LINEAR REUSABLE NEURAL BASES ARCHITECTURE

We first introduce two core principles guiding our architecture design: *neuron reuse* and *linear construction*. We then describe the two major components of our model following from these principles: the reusable neural bases and the coefficient table. Finally, we discuss the training

strategy, implementation details, and further improvement techniques, including sub-neuron reuse and parameter-sharing methods.

Throughout this section, we consider a standard residual network comprising L stacked feedforward blocks.

The Two Principles. Our model is guided by the following two principles:

- *Neuron reuse principle:* The same neuron can be reused multiple times across different depths, enabling the formation of efficient topological connections within the network.
- *Linear construction principle:* Each network block can be constructed as a linear combination of neurons.

Model Architecture. Accordingly, our proposed architecture consists of two components:

1. **A collection of reusable neural bases**, denoted by $\{\phi_j\}_{j=1}^m$, which serve as function bases for constructing different blocks.
2. **A coefficient table**, denoted by $\{\alpha_{j,l}\}_{j=1,l=1}^{m,L}$, containing scalar values that determine the contribution of each neural basis to each block.

Specifically, the residual block $R_l(x)$ at depth l is constructed via a linear combination of the shared bases $\{\phi_j\}_{j=1}^m$ with position-wise coefficients $\{\alpha_{j,l}\}_{j=1}^m$:

$$R_l(x) = \sum_{j=1}^m \alpha_{j,l} \phi_j(x).$$

Under this formulation, the same collection of neural bases is reused across all depths, while each block at a different position remains distinct due to its different coefficients.

Note that here we assume each coefficient $\alpha_{j,l}$ is real-valued and independent. In fact, one may impose additional constraints on the coefficients, such as nonnegativity or binary values, which can implicitly impact the topological connectivity of the network, yielding sparse or dense structures. Due to page limitations, we leave such investigations for future work.

Algorithm 1 Average accumulated gradients of reused neural bases.

```

1: for each parameter  $p$  in reused layers do
2:   Attach a gradient hook to  $p$ 
3:   Define hook function:  $\text{grad}, \text{depth} \mapsto \text{grad}/\text{depth}$ 
4:   Register the hook to divide accumulated gradients by depth
5: end for
```

Algorithm 2 Forward pass across all depth blocks via shared neural bases.

```

1: for  $i \leftarrow 1$  to  $L$  do
2:    $a \leftarrow \text{reused\_sensors}(x)$ 
3:    $a \leftarrow \text{Activation}(a)$ 
4:    $a \leftarrow a \odot \mathbf{c\_table}[:, i]$ 
5:    $a \leftarrow \text{reused\_responses}(a)$ 
6:    $x \leftarrow x + a$ 
7: end for
```

Training Strategy and Implementation Details. Our training approach mostly follows the standard gradient descent framework. However, because the neural bases are reused across multiple blocks, their gradients are accumulated over all positions for each optimization step. To maintain the magnitude of gradient updates consistent with standard models, we rescale the accumulated gradients of the neural bases by the number of their reused times. In contrast, the parameters of the coefficient table are updated directly without rescaling. The implementation details are summarized in Algorithm 1 and 2.

For initialization, the sensor layers of the reusable neural bases are initialized using Kaiming initialization (the default PyTorch setting), while the response layers are zero-initialized to provide a neutral starting point for stable training, following the guidance of [Bachlechner et al. \(2021\)](#); [Zhang](#)

et al. (2019). The coefficient table parameters are initialized from a standard Gaussian distribution $\mathcal{N}(0, 1)$.²

Sub-neural Level Reuse. Beyond sharing neural bases, parameter counts can be further reduced by reusing sub-neural components—specifically, sensors or responses—at a finer granularity. One simple approach is to let multiple neurons share a single response vector, yielding a more complex neuron structure with multiple sensors and one response, which we refer to as a multi-sensor neuron. Networks containing Maxout (Goodfellow et al., 2013) or SwiGLU (Shazeer, 2020) activation functions can be viewed as examples built from such multi-sensor neurons.

These multi-sensor neurons not only can reduce the parameter counts but also alleviate the dead neuron issue commonly observed in single-sensor neurons. By incorporating multiple sensors, the active domain of the neuron is expanded across the input space, enabling more efficient feature representation and more stable training dynamics.

5 DISCUSSION

Compared to classical neural networks, the reuse principle in our model introduces several new characteristics, particularly in terms of architecture shape, network compression, and coefficient energy. We discuss these characteristics below, followed by a brief discussion of the limitations.

5.1 ARCHITECTURE SHAPE

In classical neural networks, the parameter count is directly tied to the architecture shape, depending on the embedding dimension, width (hidden dimension), and depth. Designing the network architecture requires balancing the trade-offs among these dimensional factors and predefining the shape to achieve good performance. In contrast, our model reuses a shared collection of neural bases across all depths, making the total parameter count primarily dependent on width and almost decoupled from network depth. Although the coefficient table also grows linearly with depth L , its memory footprint remains negligible compared to the full-weight matrices of classical architectures. Consequently, much wider and deeper networks can be deployed under the same memory budget.

5.2 NETWORK COMPRESSION

Another interesting aspect of our model relates to network compression. Unlike classical networks, where compression is primarily achieved at the data-representation level by approximating the target function with fewer parameters, our approach introduces compression on two additional levels:

1. **Component-Level Compression:** Reusing the same neural bases across depths allows the network to capture recurrent or cyclical patterns more effectively, yielding a more parameter-efficient representation.
2. **Topological Connection Compression:** Our model enables the construction of wider and deeper networks, increasing the opportunity to discover essential topological interconnections among neurons and thereby forming networks at a high compression rate.

In alignment with the philosophy that “compression is intelligence”, highly compressed models with fewer parameters typically exhibit better robustness to noise and better generalization performance.

5.3 COEFFICIENT ENERGY AND PRUNING

Inspired by the spectral energy in Fourier analysis, we may define the energy of a neural basis via its coefficients. For instance, we can define the energy of the j -th neuron as the sum of its squared

²Although other initialization schemes are possible, simultaneously zero-initializing both the coefficient table and the response vectors should be avoided. Since these two components are connected through a bilinear structure, zero initialization would cause their gradients to vanish entirely.

coefficients across all depths:

$$E(\phi_j) = \sum_{l=1}^L \alpha_{j,l}^2.$$

This energy reflects the importance or frequency of the j -th neural basis with respect to the dataset and the network architecture, and can serve as a criterion for pruning less important neurons to further reduce memory cost.

5.4 LIMITATIONS

A primary limitation of our method is the increased training time and energy consumption compared to classical models, as block-specific weights must be dynamically computed and updated during backpropagation. Nevertheless, this computational cost is confined to the training phase; it can be eliminated during inference by precomputing and caching the effective weights for each block.

6 EXPERIMENTS

We evaluate the proposed model in two settings: an FFN-based ResNet for function approximation and a decoder-only Transformer for language modeling. All experiments are conducted on a Linux workstation equipped with an Intel Core i7 CPU, 16 GB of RAM, and an NVIDIA RTX 3090 GPU. For fair comparisons, all models use the same random seed and identical mini-batch sampling order throughout training.

6.1 EVALUATION ON AN FFN-BASED RESNET

We first evaluate the proposed model on a nonlinear regression task.

Dataset and Training Setup. The target function is the composite function $f(x) = \cos(\cos x)$, which provides a nonlinear benchmark for evaluating function-approximation ability. We uniformly sample $N = 60,000$ points from the interval $[-1, 1]$ and randomly divide them into training and test sets using a 3:1 ratio. All models are trained for 2,048 epochs with a batch size of 512. We use the mean squared error (MSE) loss and the AdamW optimizer with a learning rate of $lr = 1 \times 10^{-5}$.

Model Configuration. The baseline architecture is an FFN-based residual network consisting of $L = 8$ stacked residual blocks. Two additional projection blocks are placed at the beginning and end of the network to map the input to the embedding space and the final representation to the output space, respectively. The embedding dimension is set to $d = 64$, and the hidden dimension, corresponding to the number of neurons in each FFN block, is set to $m = 256$. To improve training stability and ensure a fair comparison, the projection blocks use the sensor-response duplex structure, with the response layers initialized to zero.

Evaluation Metrics. We evaluate the models using training and test MSE, per-epoch training time, and visual comparisons between the learned predictions and the target function. Due to space limitations, the test MSE loss results and prediction visualizations are reported in Appendix A.2. Figure 2 presents the training-loss and training-time comparisons.

Results. As shown in Figure 2(a), LRNBA achieves a convergence rate and final loss comparable to those of the standard ResNet while using only 1/7 of its parameters. It also substantially outperforms the parameter-matched *classical small* model and the baseline that repeatedly applies the same residual block across depth.

Increasing the width by a factor of four ($4m$) further accelerates convergence while achieving a comparable final loss with only half as many parameters as the standard ResNet. Increasing the depth by a factor of four ($4L$) yields even faster convergence and a lower final loss while using only 1/6 of the parameters. These results indicate that depth expansion provides greater performance gains than width expansion and that LRNBA improves parameter efficiency without compromising training stability or accuracy.

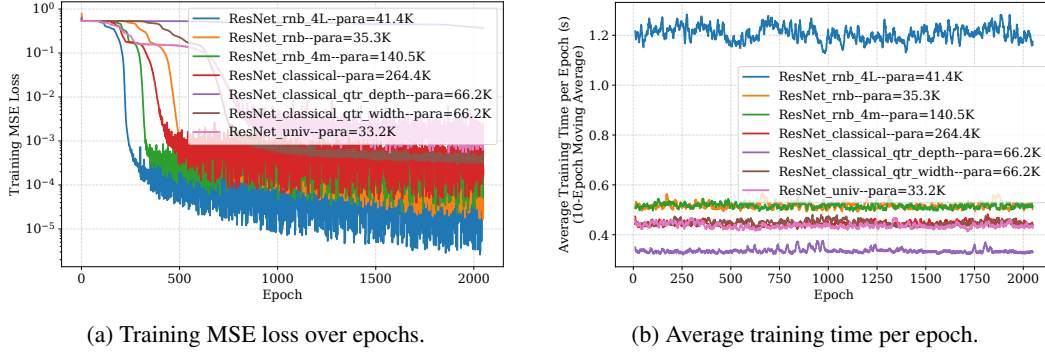


Figure 2: Comparison of FFN-based ResNet models on the nonlinear regression task. **(a)** Training MSE loss over epochs. The proposed LRNBA models achieve competitive or even faster convergence speed while using substantially fewer parameters than the standard ResNet baselines. **(b)** Smoothed training time over epochs. LRNBA introduces a modest training-time overhead because its residual blocks are dynamically constructed from reusable neural bases and depth-dependent coefficients.

As shown in Figure 2(b), LRNBA incurs a modest increase in per-epoch training time because its residual blocks are dynamically constructed from reusable neural bases and their coefficients. This overhead can be removed during inference by precomputing and caching the effective weights.

6.2 EVALUATION OF THE TRANSFORMER MODEL

To further evaluate the proposed architecture, we apply it to a decoder-only Transformer on a language-modeling task.

Dataset and Tokenization. We conduct experiments on the WikiText-2 dataset. We use a byte-level BPE tokenizer with a total vocabulary size of $2^{14} = 16,384$, including 4 special tokens (e.g., UNK, PAD, EOS, SOS). We first concatenate all sentences into a single sequence, and then randomly sample overlapping chunks of sequence length $T = 128$ tokens.

Model Configuration. For the language model, we use the standard Transformer (Vaswani et al., 2017) as the base model, with the following configuration: embedding dimension $d = 512$, depth $L = 8$, and $H = 8$ attention heads (head dimension 64). The hidden dimension of the FFN-based blocks is set to four times the embedding dimension, i.e., 2048, and no dropout is applied.

Training Setup. Each model is trained for 800 epochs using the AdamW optimizer with a learning rate of $lr = 1.5 \times 10^{-5}$. Each epoch consists of 512 training batches with a batch size of 16. For evaluation, we use 128 test batches. Although the original setup states that MSE loss is used for training, language modeling is typically optimized using cross-entropy loss; therefore, this should be verified for consistency.

Evaluation Metrics. We evaluate the models using training and test cross-entropy (CE) loss, per-epoch training time, and parameter count. Due to space limitations, the test MSE loss results and prediction visualizations are reported in Appendix A.2.

Results. As shown in Figure 6.2(a), increasing the numbers of reusable attention and FFN bases consistently accelerates convergence and reduces the final training loss. In particular, The LRNBA model with expansion factor 6 (6h6m) for both heads and FFN width converges faster and achieves lower training loss than the standard Transformer, despite using only about 3/4 as many parameters. These results demonstrate that the proposed reuse mechanism improves parameter efficiency while maintaining stable training dynamics.

As shown in Figure 6.2(b), The classical Transformer yields the highest test CE loss, while the LRNBA models demonstrate that reducing parameter size consistently lowers test loss. This observation is

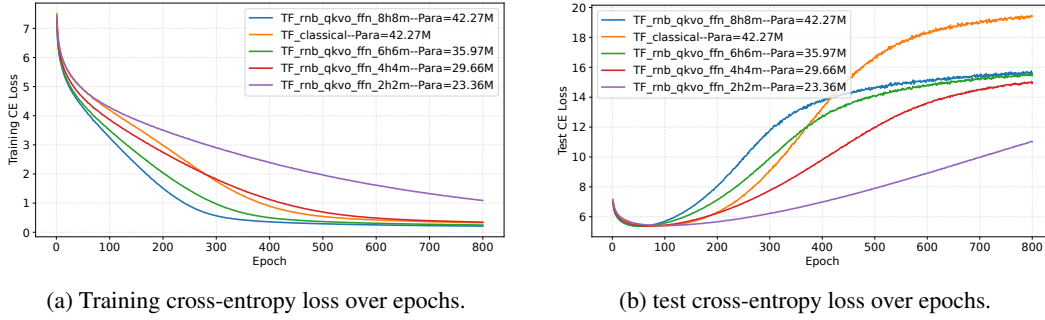


Figure 3: Comparison of decoder-only Transformer models on the WikiText-2 language-modeling task. **(a)** Training cross-entropy loss over epochs. Adding more reusable attention and FFN bases steadily improves convergence speed and lowers final training loss. The LRNBA model with expansion factor 6 (6h6m) for both heads and FFN width converges faster and achieves lower training loss than the standard Transformer, despite using only about 3/4 as many parameters. **(b)** Test cross-entropy loss over epochs. The classical Transformer yields the highest test CE loss, while the LRNBA models demonstrate that reducing parameter size consistently lowers test loss. This observation is consistent with the philosophy that compression is intelligence—the model with fewer parameters tends to achieve better generalization on test data.

consistent with the philosophy that compression is intelligence—the model with fewer parameters tends to achieve better generalization on test data.

Nevertheless, the proposed models remain substantially more parameter-efficient than the standard Transformer, making them well suited to memory-constrained applications where training time is a secondary consideration.

7 CONCLUSION

In this paper, we introduced the Linear Reusable Neural Bases Architecture (LRNBA) to improve parameter efficiency and reduce memory requirements. By decomposing feedforward neural networks into reusable neural bases and linear coefficients, we provided a geometric interpretation of neural computation through vector fields.

Guided by the principles of neuron reuse and linear construction, LRNBA model enables the construction of substantially wider and deeper networks under a limited memory budget. Our experiments on FFN-based ResNets and decoder-only Transformers demonstrate that LRNBA significantly reduces the parameter count while maintaining stable training and competitive performance. In several settings, expanding the number of reusable bases or network width and depth further improves convergence and final accuracy, highlighting the potential of neural-basis reuse for memory-constrained deep learning studies.

REFERENCES section; see below). Please note that pages should be numbered.

REFERENCES

- Thomas Bachlechner, Bodhisattwa Prasad Majumder, Henry Mao, Gary Cottrell, and Julian McAuley. Rezero is all you need: fast convergence at large depth. In Cassio de Campos and Marloes H. Maathuis (eds.), *Proceedings of the Thirty-Seventh Conference on Uncertainty in Artificial Intelligence*, volume 161 of *Proceedings of Machine Learning Research*, pp. 1352–1361. PMLR, 27–30 Jul 2021. URL <https://proceedings.mlr.press/v161/bachlechner21a.html>. 3, 6
- Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Trellis networks for sequence modeling. In *International Conference on Learning Representations (ICLR)*, 2019. 3
- Y. Bengio, P. Simard, and P. Frasconi. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166, 1994. 3

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf. 1
- Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Conference on Empirical Methods in Natural Language Processing*, 2014. URL <https://api.semanticscholar.org/CorpusID:5590763>. 3
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. 3
- Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. ISSN 0364-0213. 1, 3
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23(1), January 2022. ISSN 1532-4435. 1
- Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *Proceedings of the 40th International Conference on Machine Learning*, pp. 11398–11442, 2023. 3
- Ian Goodfellow, David Warde-Farley, Mehdi Mirza, Aaron Courville, and Yoshua Bengio. Maxout networks. In Sanjoy Dasgupta and David McAllester (eds.), *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pp. 1319–1327, Atlanta, Georgia, USA, 17–19 Jun 2013. PMLR. URL <https://proceedings.mlr.press/v28/goodfellow13.html>. 7
- David Ha, Andrew Dai, and Quoc V. Le. Hypernetworks, 2016. URL <https://arxiv.org/abs/1609.09106>. 3
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015. 4
- Kyle Helfrich, Devin Willmott, and Qiang Ye. Orthogonal recurrent neural networks with scaled cayley transform. In Jennifer Dy and Andreas Krause (eds.), *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1969–1978. PMLR, 07–09 Jul 2018. 3
- Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8): 1735–1780, November 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL <https://doi.org/10.1162/neco.1997.9.8.1735>. 3
- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS ’22*, Red Hook, NY, USA, 2022. Curran Associates Inc. ISBN 9781713871088. 1
- Michael I. Jordan. Chapter 25 - serial order: A parallel distributed processing approach. In John W. Donahoe and Vivian Packard Dorsel (eds.), *Neural-Network Models of Cognition*,

- volume 121 of *Advances in Psychology*, pp. 471–495. North-Holland, 1997. doi: [https://doi.org/10.1016/S0166-4115\(97\)80111-2](https://doi.org/10.1016/S0166-4115(97)80111-2). URL <https://www.sciencedirect.com/science/article/pii/S0166411597801112>. 1, 3
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>. 1
- Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations*, 2020. 3
- Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539. 3
- Zhiyun Lu, Vikas Sindhvani, and Tara N. Sainath. Learning compact recurrent neural networks. In *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 5960–5964, 2016. 3
- Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '21*, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450384421. doi: 10.1145/3458817.3476209. URL <https://doi.org/10.1145/3458817.3476209>. 1
- Alexander Novikov, Dmitry Podoprikin, Anton Osokin, and Dmitry Vetrov. Tensorizing neural networks, 2015. URL <https://arxiv.org/abs/1509.06569>. 3
- Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on International Conference on Machine Learning - Volume 28, ICML'13*, pp. III–1310–III–1318. JMLR, 2013. 3
- Telmo Pires, António Vilarinho Lopes, Yannick Assogba, and Hendra Setiawan. One wide feed-forward is all you need. In Philipp Koehn, Barry Haddow, Tom Kocmi, and Christof Monz (eds.), *Proceedings of the Eighth Conference on Machine Translation*, pp. 1031–1044, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.wmt-1.98. URL <https://aclanthology.org/2023.wmt-1.98/>. 3
- Maithra Raghu, Ben Poole, Jon Kleinberg, Surya Ganguli, and Jascha Sohl-Dickstein. On the expressive power of deep neural networks. In Doina Precup and Yee Whye Teh (eds.), *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 2847–2854. PMLR, 06–11 Aug 2017. URL <https://proceedings.mlr.press/v70/raghu17a.html>. 3
- Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '20*. IEEE Press, 2020. ISBN 9781728199986. 3
- Noam Shazeer. Glu variants improve transformer, 2020. URL <https://arxiv.org/abs/2002.05202>. 7
- Vikas Sindhvani, Tara Sainath, and Sanjiv Kumar. Structured transforms for small-footprint deep learning. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett (eds.), *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. URL https://proceedings.neurips.cc/paper_files/paper/2015/file/851300ee84c2b80ed40f51ed26d866fc-Paper.pdf. 3
- Shawn Tan, Yikang Shen, Zhenfang Chen, Aaron Courville, and Chuang Gan. Sparse universal transformer, 2023. URL <https://arxiv.org/abs/2310.07096>. 3

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS'17*, pp. 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964. 3, 9

Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. *arXiv preprint arXiv:2311.12424*, 2023. 3

Hongyi Zhang, Yann N. Dauphin, and Tengyu Ma. Fixup initialization: Residual learning without normalization, 2019. URL <https://arxiv.org/abs/1901.09321>. 6

A ATTENTION NEURAL BASES AND PROJECTION NEURAL BASES

A.1 ATTENTION NEURAL BASES

In this subsection, we provide a detailed description of attention neural bases.

A standard multi-head attention block with H attention heads can be written as

$$\text{Attn}(x) = \text{Concat}(A_1(x), A_2(x), \dots, A_H(x)) O,$$

where O denotes the output projection matrix and the output of the j th attention head is

$$A_j(x) = \text{softmax}\left(\frac{q_j k_j^\top}{\sqrt{d_H}}\right) v_j.$$

Here,

$$q_j = Q_j x, \quad k_j = K_j x, \quad v_j = V_j x,$$

are the query, key, and value representations associated with the j -th attention head, and d_H denotes the dimension of each head.

From the neural-basis perspective, each attention head A_j can be viewed as a nonlinear basis function that extracts a particular interaction pattern from the input sequence. Therefore, we refer to each attention head as an *attention neural basis* of the network, and the parameters (Q_j, K_j) are called the *sensor* of the attention neural basis A_j , while V_j is called the *response* of the attention neural bases. The complete attention output is then constructed by concatenating the outputs of these basis functions and combining them through the output projection O .

Similarly, we introduce scalar coefficients α_j for those heads. The resulting attention block is given by

$$\text{Attn}_\alpha(x) = \text{Concat}(\alpha_1 A_1(x), \alpha_2 A_2(x), \dots, \alpha_H A_H(x)) O,$$

where the coefficients $\{\alpha_j\}_{j=1}^H$ determine the relative coefficient of the attention heads to the final output.

A.2 PROJECTION NEURAL BASES

In this subsection, we provide a detailed description of projection neural bases, including both the derivation and visualizations.

Derivation Instead of representing each projection block as a single linear layer, we adopt a sensor–response duplex structure, using the identity function as the activation functions. This duplex structure still works as a linear projection, as it consists of two linear layers produced in sequence.

Specifically, for a composite projection block, we adopt the same notation as in the FFN-based ResNet for the sensor (S, b) and response P , and write it as

$$P(Sx + b).$$

Let m denote the hidden dimension of the block, and write the rows of S , the entries of b , and the columns of P as

$$S^\top = [s_1, s_2, \dots, s_m], \quad b^\top = [b_1, b_2, \dots, b_m], \quad P = [p_1, p_2, \dots, p_m],$$

where s_j is the j -th row of S , b_j is the j -th entry of b , and p_j is the j -th column of P .

For each $j = 1, 2, \dots, m$, we define the function $\phi_j(x)$ as:

$$\phi_j(x) := (\langle x, s_j \rangle + b_j) \cdot p_j,$$

where the activation function $\sigma(x)$ is replaced with identify functions.

The projection block can be rewritten as a summation of such functions:

$$P(Sx + b) = \sum_{j=1}^m (\langle x, s_j \rangle + b_j) p_j = \sum_{j=1}^m \phi_j(x). \quad (3)$$

This formulation reveals that the projection block can be further decomposed into a collection of more elementary functions $\{\phi_j\}_{j=1}^m$.

Based on this decomposition, we refer to each function ϕ_j as a *projection neuron* or *projection neural basis* of the network, and similarly, we call the parameters (s_j, b_j) the *sensor* of neuron ϕ_j and p_j the *response* of neuron ϕ_j .

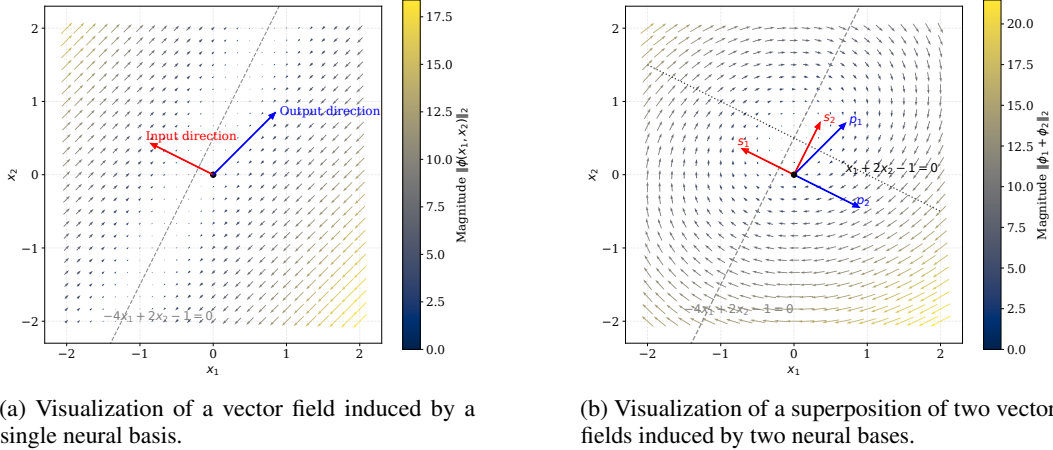
An interesting observation is that each projection neural basis actually corresponds to a rank-one linear(affine) transformation from the input space to the output space, since the expression can be expanded as

$$\phi_j(x) = (\langle x, s_j \rangle + b_j) \cdot p_j = p_j s_j^\top x + p_j b_j,$$

where $p_j s_j^\top$ is a rank-one matrix and $p_j b_j$ serves as a bias of the linear transformation.

Visualizations We also provide a visualization of the projection neural basis via vector fields. For simplicity, we adopt the same parameter values as those used in the earlier visualizations (see Figure 4).

For the j -th projection neuron, as illustrated in Figure 4a, the sensor vector s_j and scalar b_j specifies the direction and bias of the input vector field, and vector p_j determine the response vector in the output vector field. These parameters are connected through a outer product between value $\langle x, s_j \rangle + b_j$ and the response vector p_j . Consequently, a residual feedforward block can be actually interpreted as a superposition of vector fields, as shown in Figure 4b.



(a) Visualization of a vector field induced by a single neural basis.

(b) Visualization of a superposition of two vector fields induced by two neural bases.

Figure 4: Geometric interpretation of projection neural bases from the perspective of vector fields. **(a)** Visualization of a single projection neural basis, defined by the function $\phi(x) = (\langle x, s \rangle + b)p$ with parameters $s = (-4, 2)$, $b = -1$, and $p = (1, 1)$. The red and blue arrows indicate the directions of the input and output vector fields, respectively, while the dashed line marks the decision boundary between the active and inactive domains. The small arrows on the grid illustrate the push-forward effect for the corresponding input positions induced by this basis function. **(b)** A superposition of two vector fields, where the first projection neural basis is identical to that shown in **(a)** and the second projection neural basis has parameters $s_2 = (1, 2)$, $b_2 = -1$, and $p_2 = (2, -1)$. Note that a single neuron induces only a directional vector effect, whereas the combination of two neurons can generate a field exhibiting rotational dynamics, as illustrated in **(b)**.

ADDITIONAL EXPERIMENTAL RESULTS OF FFN-BASED RESNET

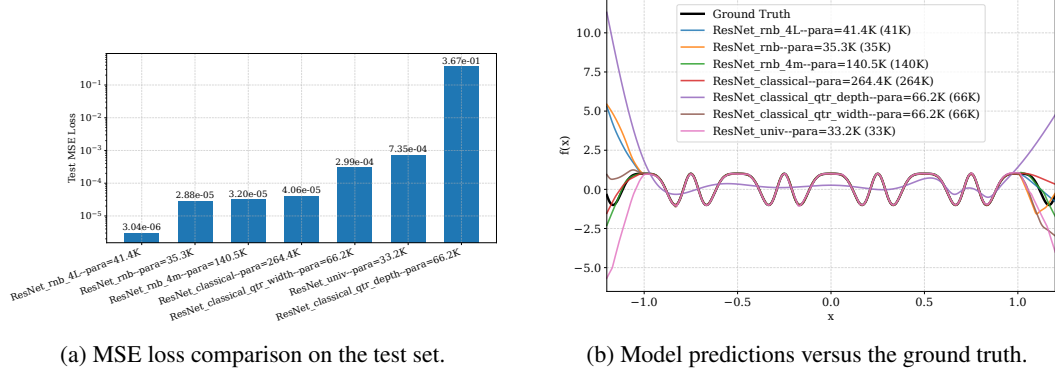


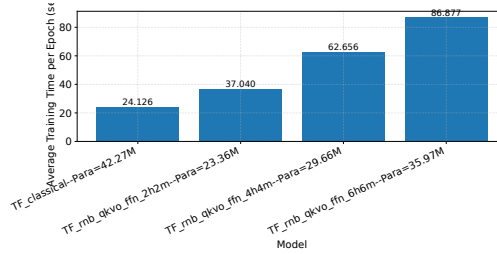
Figure 5: Test-set performance of LRNBA and classical ResNet variants. (a) Test MSE loss comparison; (b) Visualization of model predictions against ground truth.

Results. As shown in Figure 5(a), the test-set results are consistent with the training-set observations. The LRNBA model with a fourfold increase in depth achieves the lowest MSE, whereas the classical model with one-quarter of the original depth produces the highest loss. The LRNBA model also outperforms the baseline that directly reuses the same residual block.

These results support two main observations: depth expansion provides a larger reduction in test error than width expansion, and the proposed reuse mechanism improves parameter efficiency while preserving strong generalization performance.

As shown in Figure 5(b), all models except the one-quarter-depth classical variant closely approximate the ground-truth function. This comparison further highlights the importance of depth for accurate function approximation and demonstrates that LRNBA enables deeper networks under a limited parameter budget.

ADDITIONAL EXPERIMENTAL RESULTS OF TRANSFORMER MODELS



(a) Training cross-entropy loss over epochs.

Figure 6: Average training time per epoch. Because of parallel computation, the training and inference times are primarily determined by the network depth and are relatively insensitive to the network width. In practice, however, we observe that the computational time may also increase approximately with the number of neural bases. This growth may be related to the limited on-chip SRAM capacity of GPUs and could potentially be alleviated by larger on-chip memory or more efficient memory-management techniques.

Results. As shown in Figure 6, the reduction in memory usage comes with additional training-time overhead. Since the attention and FFN blocks are dynamically constructed from reusable neural bases and their associated coefficients, the average training time increases approximately linearly with the number of bases.

